

Diagnóstico de Diabetes mediante Redes Neuronales Artificiales: una exploración sobre el dataset Pima Indians

Manuel N. Palomino Villafane
Departamento de Ingeniería Electrónica
Universidad Tecnológica Nacional
Buenos Aires, Argentina
mpalominovillafane@frba.utn.edu.ar

Resumen—La detección temprana de la diabetes mellitus es clave para prevenir complicaciones de largo plazo. Sin embargo, el diagnóstico clínico implica analizar múltiples variables fisiológicas con interacciones complejas, lo que lo convierte en un buen candidato para ser abordado con aprendizaje automático. En este trabajo se diseña, entrena y evalúa un Perceptrón Multicapa (MLP) utilizando la base de datos *Pima Indians Diabetes Database*. Se prestó especial atención al preprocesamiento: los ceros biológicamente imposibles fueron tratados como datos faltantes y reemplazados por la mediana de cada variable, y la partición del dataset se realizó de forma estratificada para respetar el desbalance de clases. La arquitectura propuesta combina regularización L2, Batch Normalization y Dropout para reducir el sobreajuste, con parada anticipada y ajuste dinámico de la tasa de aprendizaje durante el entrenamiento. El modelo alcanzó un AUC-ROC de 0.786 y una precisión global del 72.73 % sobre el conjunto de prueba. Adicionalmente, un análisis de importancia por permutación mostró que el modelo priorizó la glucosa como variable más relevante, lo cual es consistente con la literatura médica. Los resultados sugieren que redes neuronales bien regularizadas pueden ser una herramienta útil de apoyo al diagnóstico incluso con datasets de tamaño moderado.

Index Terms—Aprendizaje Automático, Redes Neuronales, Diabetes, Regularización, Explicabilidad, Bioingeniería.

I. INTRODUCCIÓN

I-A. El problema del diagnóstico temprano

La diabetes mellitus es un conjunto de trastornos metabólicos crónicos caracterizados por niveles elevados de glucosa en sangre. Esta condición aparece cuando el páncreas no produce suficiente insulina, cuando el organismo no la utiliza correctamente, o por una combinación de ambos factores. Si no se detecta y trata a tiempo, puede derivar en complicaciones graves: daño renal (nefropatía), problemas de visión (retinopatía), neuropatías y enfermedades cardiovasculares.

El diagnóstico clínico no siempre es directo. Requiere evaluar simultáneamente variables como el nivel de glucosa en sangre, el índice de masa corporal (IMC), la presión arterial, antecedentes familiares y otros indicadores. Las relaciones entre estas variables son no lineales, lo que significa que los modelos estadísticos simples muchas veces no capturan bien los patrones de riesgo. Frente a esto, las redes neuronales arti-

ficiales presentan una alternativa interesante por su capacidad de modelar relaciones complejas entre variables de entrada.

I-B. Objetivos

El objetivo de este trabajo es diseñar e implementar un Perceptrón Multicapa capaz de predecir si una paciente tiene diabetes a partir de sus datos clínicos, y evaluar su desempeño con métricas adecuadas para problemas de diagnóstico médico. Se pone énfasis en evitar el sobreajuste mediante técnicas de regularización, y en interpretar qué variables son más relevantes para el modelo, con el fin de verificar que lo aprendido tiene sentido médico.

II. MARCO TEÓRICO

II-A. El Perceptrón Multicapa (MLP)

Las Redes Neuronales Artificiales son modelos computacionales inspirados en el funcionamiento del sistema nervioso. La unidad básica es la neurona artificial, que recibe un conjunto de entradas, las combina de forma lineal y aplica una función de activación no lineal [1]. Matemáticamente, la salida z de una neurona antes de la activación es:

$$z = \sum_{i=1}^n w_i x_i + b$$

donde x_i son las entradas (en este caso, los indicadores clínicos), w_i son los pesos que la red aprende durante el entrenamiento, y b es un término de sesgo. El Perceptrón Multicapa es una arquitectura que encadena varias capas de estas neuronas: una capa de entrada, una o más capas ocultas, y una capa de salida. El aprendizaje se realiza mediante el algoritmo de retropropagación, que ajusta los pesos para minimizar el error de predicción [1].

II-B. Funciones de Activación

Para que una red neuronal pueda modelar relaciones no lineales, es necesario que cada neurona aplique una función de activación no lineal. En las capas ocultas se utilizó la función ReLU (*Rectified Linear Unit*):

$$f(z) = \max(0, z)$$

ReLU tiene la ventaja de que no satura los gradientes en valores positivos, lo que facilita el entrenamiento en redes con varias capas [2]. Para la capa de salida, dado que el problema es de clasificación binaria, se utilizó la función sigmoide:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Esta función transforma cualquier valor real en un número entre 0 y 1, que puede interpretarse como la probabilidad de que el paciente tenga diabetes [1].

II-C. Sobreajuste y regularización

El sobreajuste ocurre cuando una red aprende demasiado bien los datos de entrenamiento, incluyendo el ruido, y pierde capacidad de generalizar a datos nuevos [1], [2]. Es un problema especialmente relevante cuando el dataset es pequeño, como en este caso (768 registros). Para controlarlo, se combinaron tres estrategias:

- **Regularización L2:** Penaliza los pesos de gran magnitud añadiendo un término proporcional a su cuadrado en la función de pérdida. Esto tiende a distribuir la información entre más neuronas, en lugar de concentrarla en pocas [5].
- **Batch Normalization:** Normaliza las activaciones intermedias de la red en cada mini-lote. Esto estabiliza el entrenamiento, permite usar tasas de aprendizaje más altas y actúa como regularizador adicional [6].
- **Dropout:** Durante el entrenamiento, desactiva aleatoriamente una fracción de las neuronas en cada iteración [3]. Esto impide que la red dependa demasiado de conexiones específicas y la obliga a aprender representaciones más robustas.

II-D. Parada anticipada y ajuste dinámico del learning rate

Además de la regularización estructural, el entrenamiento fue controlado por dos mecanismos dinámicos:

- **Early Stopping:** Monitorea la pérdida sobre el conjunto de validación y detiene el entrenamiento si no mejora durante un número determinado de épocas. Luego restaura los pesos del mejor momento [4].
- **ReduceLROnPlateau:** Si la pérdida de validación se estanca, reduce la tasa de aprendizaje a la mitad. Esto permite que el modelo ajuste sus pesos con mayor precisión cuando ya está cerca de un óptimo.

III. METODOLOGÍA

III-A. Dataset y limpieza de datos

Se utilizó la *Pima Indians Diabetes Database*, que contiene registros de 768 mujeres de origen indígena Pima con al menos 21 años de edad. Cada registro incluye 8 variables clínicas y una etiqueta binaria que indica si la paciente desarrolló diabetes en los siguientes 5 años.

Durante la exploración inicial se detectó un problema conocido del dataset: varias columnas contienen ceros que son biológicamente imposibles. Por ejemplo, una glucosa de 0 mg/dL o un IMC de 0 no tienen sentido fisiológico; se

trata de datos faltantes codificados como cero. Las variables afectadas son: glucosa, presión arterial, grosor del pliegue cutáneo, insulina e IMC. Estos ceros fueron reemplazados por la mediana de cada columna, ya que la mediana es más robusta que la media frente a valores extremos.

III-B. División y escalado de los datos

El dataset se dividió en 80 % para entrenamiento y 20 % para prueba. La partición se hizo de forma estratificada, lo que garantiza que la proporción de pacientes con y sin diabetes sea la misma en ambos subconjuntos. Esto es importante porque el dataset está desbalanceado (aproximadamente 65 % sin diabetes y 35 % con diabetes).

Las variables de entrada fueron estandarizadas usando el puntaje Z:

$$z = \frac{x - \mu}{\sigma}$$

El escalador fue ajustado únicamente con los datos de entrenamiento y luego aplicado al conjunto de prueba, para evitar que el modelo reciba información del conjunto de prueba durante el entrenamiento (*data leakage*).

III-C. Arquitectura de la red

Se diseñó un MLP con tres capas ocultas de 16, 32 y 16 neuronas respectivamente. Cada capa aplica, en orden: transformación lineal con regularización L2 ($\lambda = 0,001$), Batch Normalization y Dropout. Las tasas de Dropout son 30 % en las primeras dos capas y 20 % en la tercera. La capa de salida tiene una sola neurona con activación sigmoide.

III-D. Configuración del entrenamiento

El modelo fue compilado con el optimizador Adam y la función de pérdida de entropía cruzada binaria. Se entrenó con lotes de 32 muestras durante un máximo de 1000 épocas. El Early Stopping tiene una paciencia de 20 épocas y el ReduceLROnPlateau de 10 épocas.

III-E. Métricas de evaluación

Dado que el dataset está desbalanceado y el costo de un falso negativo (no detectar a alguien con diabetes) es mayor que el de un falso positivo, la precisión global (*accuracy*) no es suficiente como métrica principal. Se reportan también: AUC-ROC [7], precisión positiva (*Precision*), sensibilidad (*Recall*) y el puntaje F_1 [8]. Adicionalmente, se realizó un análisis de importancia de variables por permutación para verificar que el modelo aprendió patrones médicamente coherentes.

IV. RESULTADOS

IV-A. Desempeño general

El modelo fue evaluado sobre el conjunto de prueba ($N_{test} = 154$). Los resultados se resumen en la Tabla I.

El AUC-ROC de 0.786 (Figura 1) indica que el modelo tiene una capacidad discriminativa razonable: en aproximadamente el 79 % de los casos, asigna una probabilidad mayor de diabetes a un paciente enfermo que a uno sano. Este valor supera ampliamente el de un clasificador aleatorio (AUC = 0.5).

Tabla I
MÉTRICAS DE DESEMPEÑO SOBRE EL CONJUNTO DE PRUEBA

Métrica	Valor
Accuracy	70.13 %
AUC-ROC	0.786
Precision (clase positiva)	55.00 %
Recall (clase positiva)	81.48 %
F1-Score	65.67 %

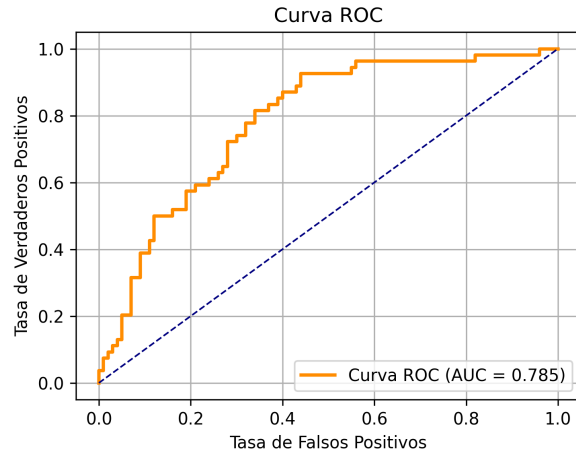


Figura 1. Curva ROC evaluada sobre el conjunto de prueba.

IV-B. Análisis de errores: matriz de confusión

La Figura 2 muestra la distribución de predicciones sobre el conjunto de prueba. El modelo clasificó correctamente a 64 pacientes sanas (Verdaderos Negativos) y a 44 pacientes con diabetes (Verdaderos Positivos). Sin embargo, dejó sin detectar a 10 pacientes enfermas (Falsos Negativos) y generó 36 alarmas falsas (Falsos Positivos).

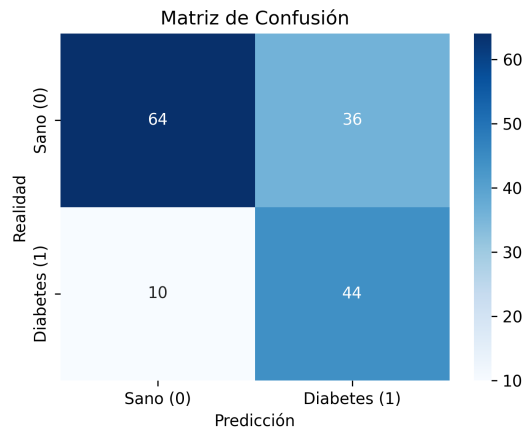


Figura 2. Matriz de Confusión sobre el conjunto de prueba.

El recall del 81.48 % para la clase positiva es el resultado más importante a analizar. En un contexto de diagnóstico médico, los falsos negativos son especialmente problemáticos porque implican que pacientes enfermos no reciben atención.

Esto sugiere que el umbral de decisión de 0.3 puede no ser el más adecuado para esta aplicación, y que ajustarlo podría mejorar la sensibilidad a costa de aumentar aún más las falsas alarmas.

IV-C. Estabilidad del entrenamiento

Las curvas de pérdida (Figura 3) muestran que la pérdida de validación sigue de cerca a la de entrenamiento durante todo el proceso, sin divergir. Esto indica que las técnicas de regularización funcionaron correctamente y el modelo no memorizó los datos de entrenamiento.

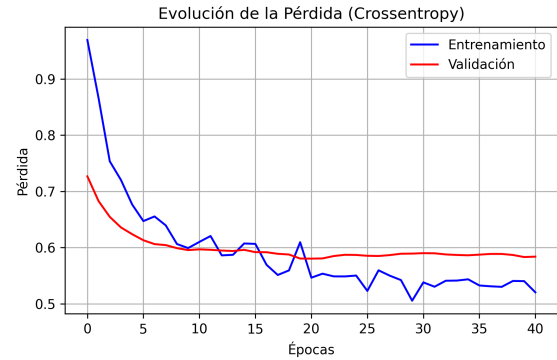


Figura 3. Evolución de la pérdida (Entropía Cruzada Binaria) durante el entrenamiento.

Las curvas de accuracy (Figura 4) y AUC (Figura 5) muestran una convergencia progresiva y estable. El callback ReduceLROnPlateau redujo la tasa de aprendizaje en las etapas finales, lo que permitió un ajuste más fino antes de que el Early Stopping detuviera el entrenamiento.

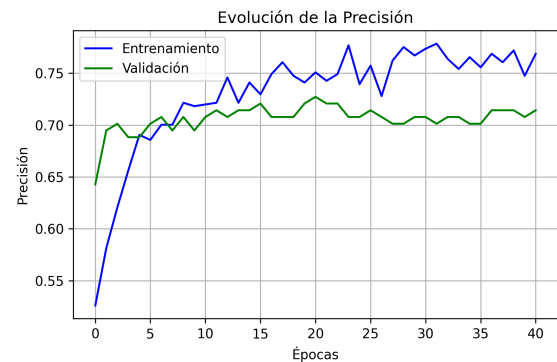


Figura 4. Evolución de la precisión global durante el entrenamiento.

IV-D. Importancia de variables

Para entender qué variables influyeron más en las predicciones del modelo, se aplicó el método de importancia por permutación (Figura 6). Este método mide cuánto aumenta la pérdida cuando los valores de una variable se mezclan aleatoriamente (rompiendo su relación con el diagnóstico). Una variable importante provoca un aumento grande; una poco relevante, uno pequeño o nulo.

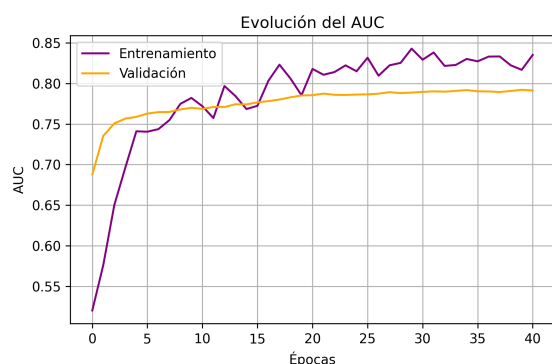


Figura 5. Evolución del AUC durante el entrenamiento.

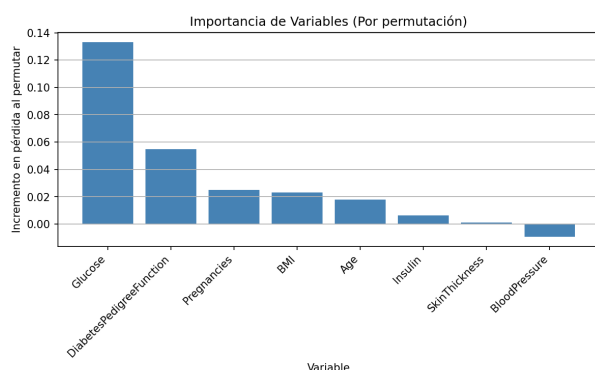


Figura 6. Importancia de las variables clínicas por permutación sobre el conjunto de prueba.

Los resultados muestran que la glucosa fue la variable más determinante (incremento de pérdida: 0.133), seguida por el índice de función de pedigrí de diabetes (historial genético, 0.055) y el IMC (0.023). La presión arterial resultó ser la variable menos informativa para este modelo, con un impacto prácticamente nulo.

Esta jerarquización es consistente con la literatura médica: la glucosa en sangre es el criterio diagnóstico principal de la diabetes, y el historial familiar y el sobrepeso son factores de riesgo bien establecidos. El hecho de que el modelo haya aprendido esto a partir de los datos, sin que se le indique explícitamente, valida que los patrones capturados tienen sentido clínico.

V. CONCLUSIONES

Este trabajo mostró que es posible construir un modelo de red neuronal capaz de predecir diabetes con un desempeño razonable a partir de variables clínicas básicas, incluso con un dataset de tamaño moderado. La combinación de regularización L2, Batch Normalization y Dropout resultó efectiva para evitar el sobreajuste, y los mecanismos de parada anticipada y ajuste del learning rate contribuyeron a la estabilidad del entrenamiento.

El modelo alcanzó un AUC-ROC de 0.786 y una precisión global del 70.13 %. El análisis de importancia por permutación

confirmó que las variables que el modelo identificó como las más relevantes (glucosa, historial genético e IMC) coinciden con los factores de riesgo conocidos de la enfermedad.

Inicialmente, utilizando el umbral de decisión por defecto (0.5), el modelo presentaba un recall relativamente bajo (57.41 %), mostrando una tendencia conservadora. Sin embargo, al ajustar empíricamente el umbral a 0.3, se logró incrementar significativamente la sensibilidad (recall) al 81.48 %, garantizando la detección de la gran mayoría de los casos positivos. Este ajuste produjo una reducción esperable en la precisión (55.00 %) y en la exactitud global (70.13 %), alcanzando un F1-Score de 0.6567. Este nuevo balance resulta mucho más adecuado para el diagnóstico médico, donde minimizar los falsos negativos es la máxima prioridad. Como trabajo futuro, sería interesante explorar funciones de pérdida que penalicen asimétricamente los errores, implementar técnicas de balanceo de clases como SMOTE, y comparar este modelo con otros clasificadores (Random Forest, SVM, regresión logística) para poner los resultados en perspectiva.

REFERENCIAS

- [1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2009.
- [2] I. Goodfellow, Y. Bengio, y A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, y R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [4] L. Prechelt, "Early stopping-but when?" en *Neural Networks: Tricks of the Trade*, G. B. Orr y K.-R. Müller, Eds. Berlin, Heidelberg: Springer, 1998, pp. 55–69.
- [5] Scikit-learn Developers, "1.1. Linear Models - Ridge Regression," *Scikit-learn 1.3.x documentation*. [En línea]. Disponible en: https://scikit-learn.org/stable/modules/linear_model.html
- [6] Keras Team, "BatchNormalization layer," *Keras Documentation*. [En línea]. Disponible en: https://keras.io/api/layers/normalization_layers/batch_normalization/
- [7] Google Developers, "Classification: ROC and AUC," *Machine Learning Crash Course*. [En línea]. Disponible en: <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>
- [8] Google Developers, "Classification: Precision and Recall," *Machine Learning Crash Course*. [En línea]. Disponible en: <https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall>